Lecture 11:

Feature Selection



Markus Hohle

University of California, Berkeley

Data Science for Scientific Computing

MSSE 277A, 3 Units

Berkeley MSSE
MOLECULAR SCIENCE & SOFTWARE ENGINEERING

Syllabus

Lecture 1: Introduction to Data Science

Lecture 2: Data Sampling and Probability, Pandas — data acquisition and analysis

Lecture 3: Exploratory Data Analysis (EDA), and Regex Part I

Lecture 4: Exploratory Data Analysis (EDA), and Regex Part II

Lecture 5: Introduction to SQL

Lecture 6: Feature Analysis, Engineering and Encoding — feature Selection and Analysis

Lecture 7: PCA, LDA and Correlation, Dimension Reduction

Lecture 8: Advanced Visualization Tools (UMAP, T-SNE) in Python

project 1 — Lecture 9: Introduction to linear and logistic regression; OLS

Lecture 10: Avoiding Overfitting and Regularization (L1 & L2) — modelling

Lecture 11: Feature Selection via Regression Analysis

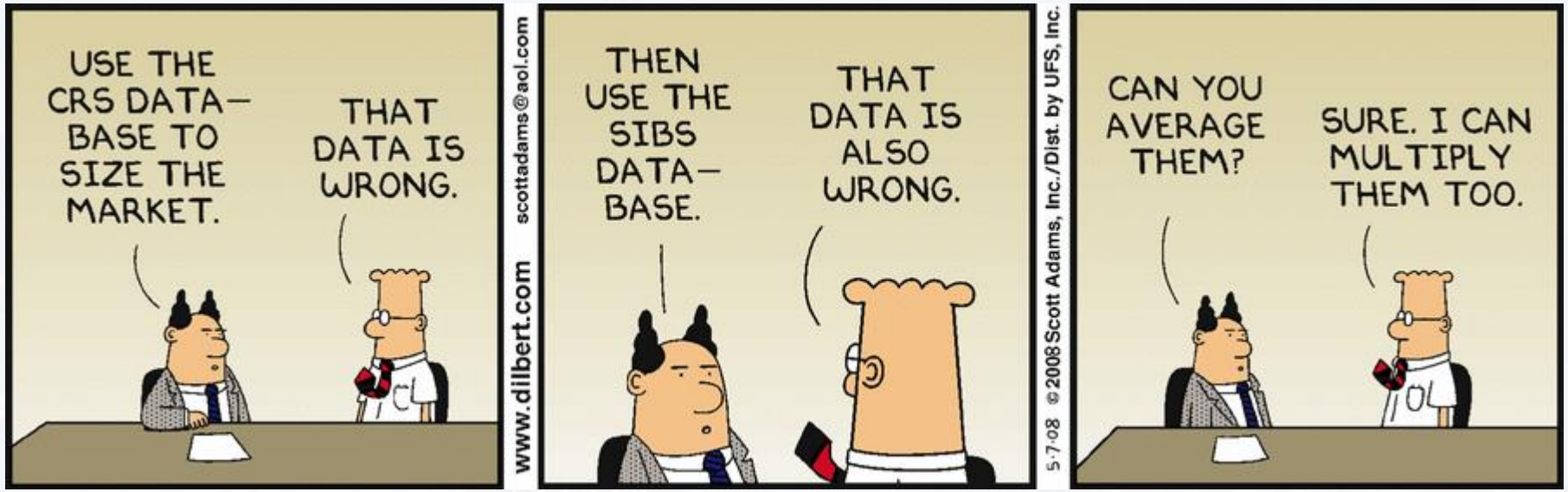
Lecture 12: Gradient Descent

Lecture 14: Clustering and Classification: KNN, K-Mean, SVM

Lecture 14: Clustering and Classification: Trees, GMM

project 2 — Lecture 15: Feature Selection via ANNs ("leave-one-out", Accuracy Drop & Entropy)

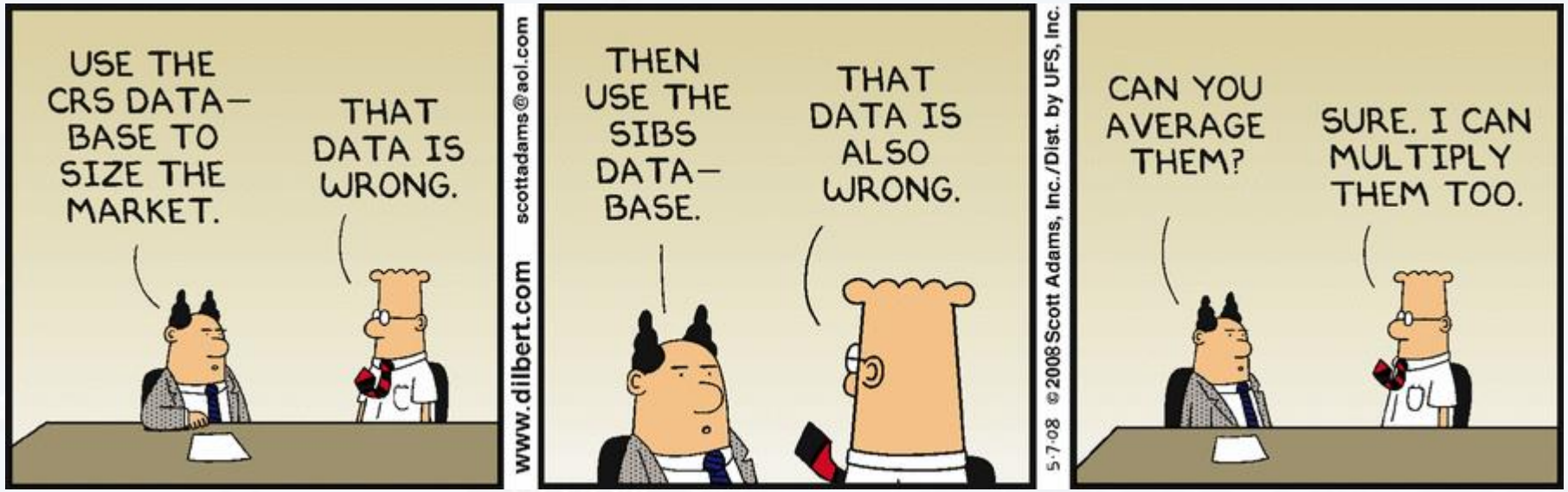
Feature Selection



Outline

- **More About p-values**
- **Feature Selection**
  - via L1 and L2
  - via p-values
  - back to PCA

Feature Selection



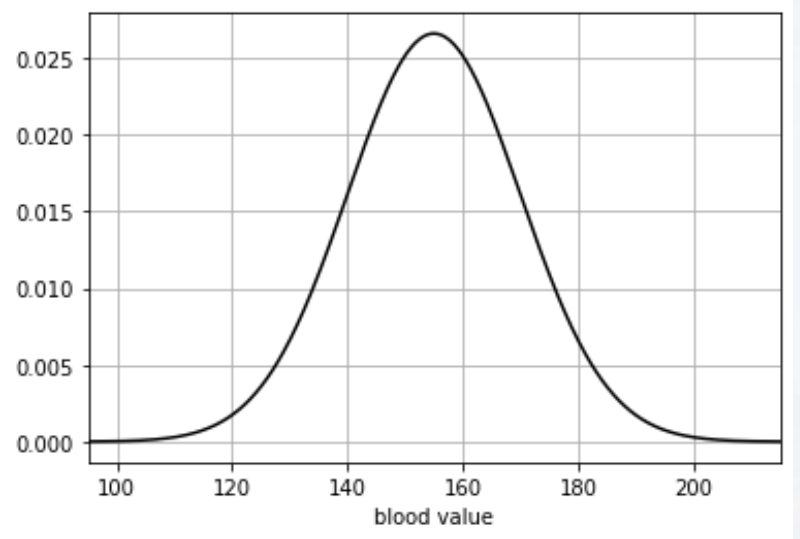
Outline

- **More About p-values**
- Feature Selection
  - via L1 and L2
  - via p-values
  - back to PCA

standard frequentist way (not Bayesian):

1) assume a **likelihood function *L* as your model**, aka **null hypothesis H_0**
2) take a datapoint x_0
3) calculate the ***probability P*** given ***L*** i. e. **given H_0** is true, **that x_0 has this value** or a **more extreme value**
4) accepting or rejecting H_0 based on ***P*** and the threshold α

example: 1) a healthy person has a blood value that follows a **normal distribution** with $\mu = 155, \sigma = 15$, i. e. $\boldsymbol{H_0: N(\mu = 155, \sigma = 15)}$



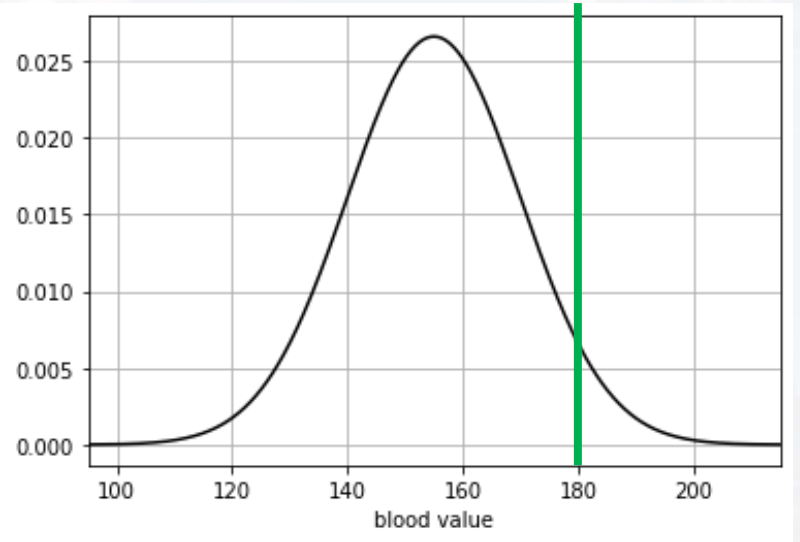
standard frequentist way (not Bayesian):

1) assume a **likelihood function *L* as your model**, aka **null hypothesis H_0**
2) take a datapoint x_0
3) calculate the ***probability P*** given ***L*** i. e. **given H_0** is true, **that x_0 has this value** or a **more extreme value**
4) accepting or rejecting H_0 based on ***P*** and the threshold α

example:

1) a healthy person has a blood value that follows a **normal distribution** with $\mu = 155$, $\sigma = 15$, i. e. $H_0: N(\mu = 155, \sigma = 15)$

2) a patient has the value x_0 **= 180**



standard frequentist way (not Bayesian):

1) assume a **likelihood function *L* as your model**, aka **null hypothesis H_0**
2) take a datapoint x_0
3) calculate the ***probability P*** given ***L*** i. e. **given H_0** is true, **that x_0 has this value** or a **more extreme value**
4) accepting or rejecting H_0 based on ***P*** and the threshold α

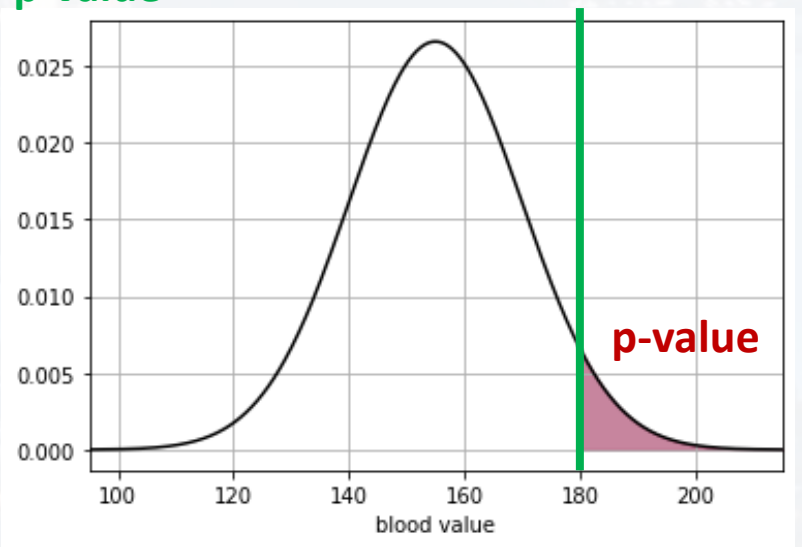
example:

1) a healthy person has a blood value that follows a **normal distribution** with

$\mu = 155, \sigma = 15$, i. e. $H_0: N(\mu = 155, \sigma = 15)$

2) a patient has the value $x_0 = 180$

3) probability $P(x \geq 180 | H_0) = 0.048$ called **p-value**



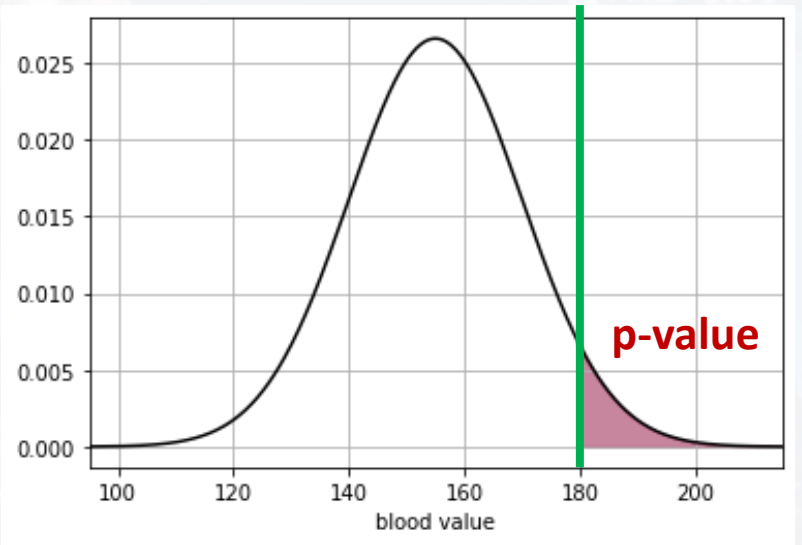
standard frequentist way (not Bayesian):

1) assume a **likelihood function *L* as your model**, aka **null hypothesis H_0**
2) take a datapoint x_0
3) calculate the ***probability P*** given ***L*** i. e. **given H_0** is true, **that x_0 has this value** or a **more extreme value**
4) accepting or rejecting H_0 based on ***P*** and the threshold α

example:

1) a healthy person has a blood value that follows a **normal distribution** with $\mu = 155, \sigma = 15$, i. e. $H_0: N(\mu = 155, \sigma = 15)$
2) a patient has the value $x_0 = 180$
3) probability $P(x \geq 180 | H_0) = 0.048$ called **p-value**
4) for **α = 0.05**, H_0 is **rejected**, i. e. patient is **not healthy**

→ alternative hypothesis H_1: **not healthy**



example:

1) a healthy person has a blood value that follows a **normal distribution** with

$\mu = 155, \sigma = 15$, i. e. $\boldsymbol{H_0: N(\mu = 155, \sigma = 15)}$

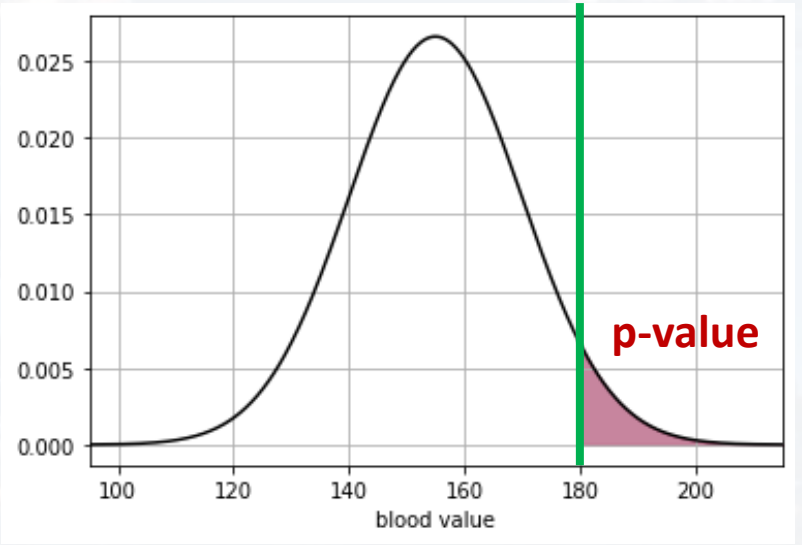
2) a patient has the value $\boldsymbol{x_0}$ **= 180**

3) probability $\boldsymbol{P(x \geq 180 | H_0) = 0.048}$ called **p-value**

4) for **α = 0.05**, H_0 is **rejected**, i. e. patient is **not healthy**

→ alternative hypothesis $\boldsymbol{H_1}$: **not healthy**

We just performed a so-called Z – Test
(comparing one value to a normal distribution)



standard frequentist way (not Bayesian):

1) assume a **likelihood function *L* as your model**, aka **null hypothesis H_0**
2) take a datapoint x_0
3) calculate the ***probability P*** given ***L*** i. e. **given H_0** is true, **that x_0 has this value** or a **more extreme value**
4) accepting or rejecting H_0 based on ***P*** and the threshold α

caveats:

- the model ***L*** for H_0 has to **be known**
- the p-value $P(x \geq x_0|H_0)$ **does not** tell if H_0 is true or not
- the p-value $P(x \geq x_0|H_0)$ **does not** tell which hypothesis is more likely
- the p-value just gives $P(x \geq x_0|H_0)$
- the threshold α for accepting/rejecting H_0 is **arbitrary**
- we are aiming on disproving a hypothesis, by assuming it is true, **without** leading it to a contradiction

- assuming H_0 is always true
- taking many samples
- running **any** non-Bayesian statistical test on each sample
- deriving a p-value each time

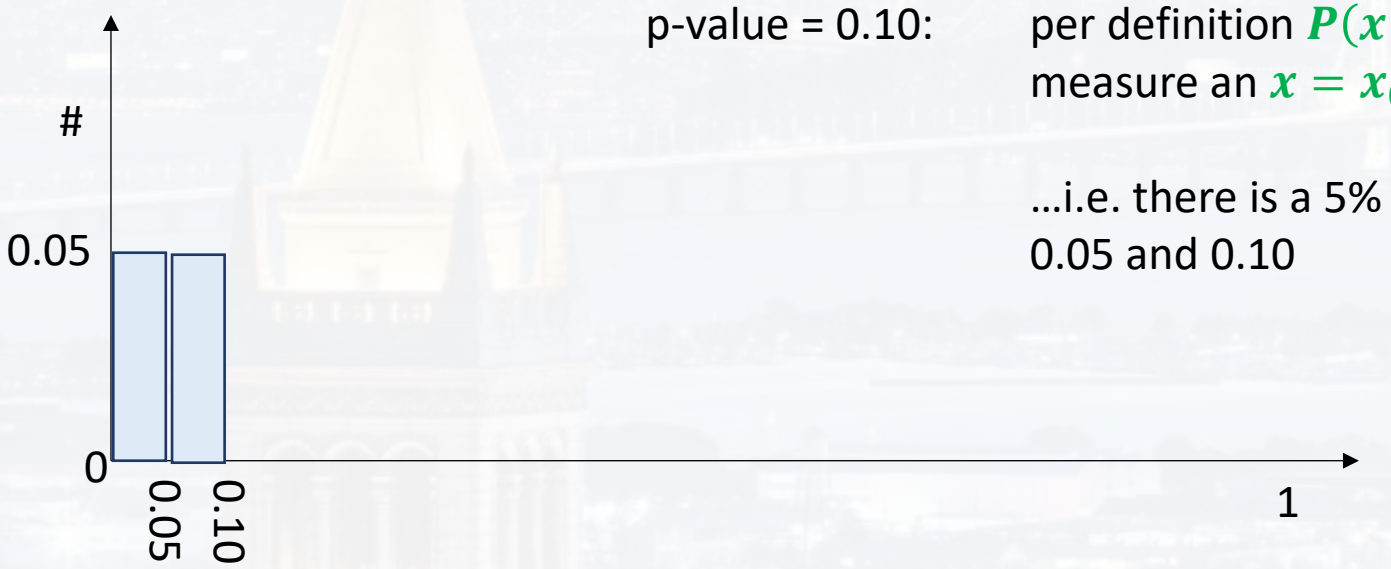
p-value $\boldsymbol{P(x \geq x_0|H_0)}$

→ what does a **p-value histogram** look like if H_0 is **always** true?

p-value = 0.05: per definition $\boldsymbol{P(x \geq x_0|H_0)} = 5\%$ of the times we measure an $\boldsymbol{x = x_0}$, which leads to a p-value ≤ 0.05

p-value = 0.10: per definition $\boldsymbol{P(x \geq x_0|H_0)} = 10\%$ of the times we measure an $\boldsymbol{x = x_0}$, which leads to a p-value ≤ 0.10

…i.e. there is a 5% chance to have a p-value between 0.05 and 0.10



- assuming H_0 is always true
- taking many samples
- running **any** non-Bayesian statistical test on each sample
- deriving a p-value each time

p-value $\boldsymbol{P(x \geq x_0|H_0)}$

→ what does a **p-value histogram** look like if H_0 is **always** true?

p-value = 0.15: per definition $\boldsymbol{P(x \geq x_0|H_0)} = 15\%$ of the times we measure an $\boldsymbol{x = x_0}$, which leads to a p-value ≤ 0.15

…i.e. there is a 5% chance to have a p-value between 0.10 and 0.15



- assuming H_0 is always true
- taking many samples
- running **any** non-Bayesian statistical test on each sample
- deriving a p-value each time

p-value $\boldsymbol{P(x \geq x_0|H_0)}$

→ what does a **p-value histogram** look like if H_0 is **always** true?

p-value = 0.20: per definition $\boldsymbol{P(x \geq x_0|H_0)} = 20\%$ of the times we measure an $\boldsymbol{x = x_0}$, which leads to a p-value ≤ 0.20

…i.e. there is a 5% chance to have a p-value between 0.15 and 0.20



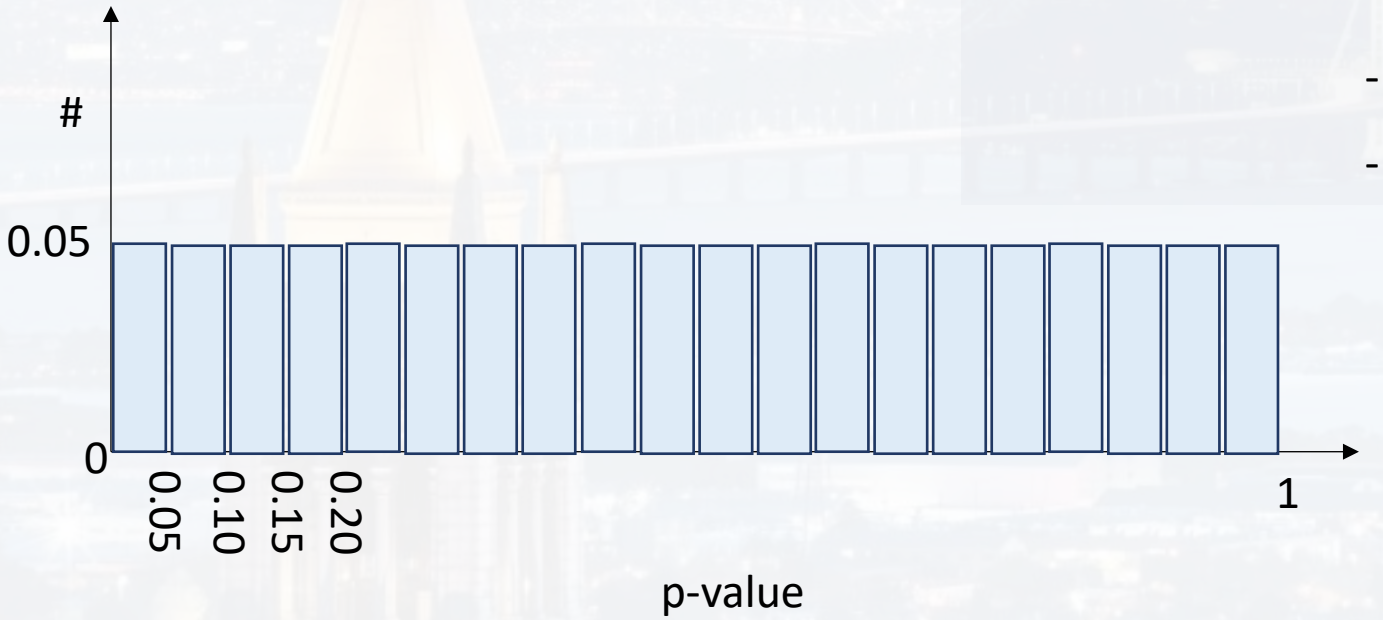
- assuming H_0 is always true
- taking many samples
- running **any** non-Bayesian statistical test on each sample
- deriving a p-value each time

p-value $\boldsymbol{P(x \geq x_0 | H_0)}$

→ what does a **p-value histogram** look like if H_0 is **always** true?

if H_0 is **always** true:

- the p-value distribution is flat!
- regardless of the test
- regardless of the data
- it is **the definition** of the p-value



- assuming H_0 is always true
- taking many samples
- running **any** non-Bayesian statistical test on each sample
- deriving a p-value each time

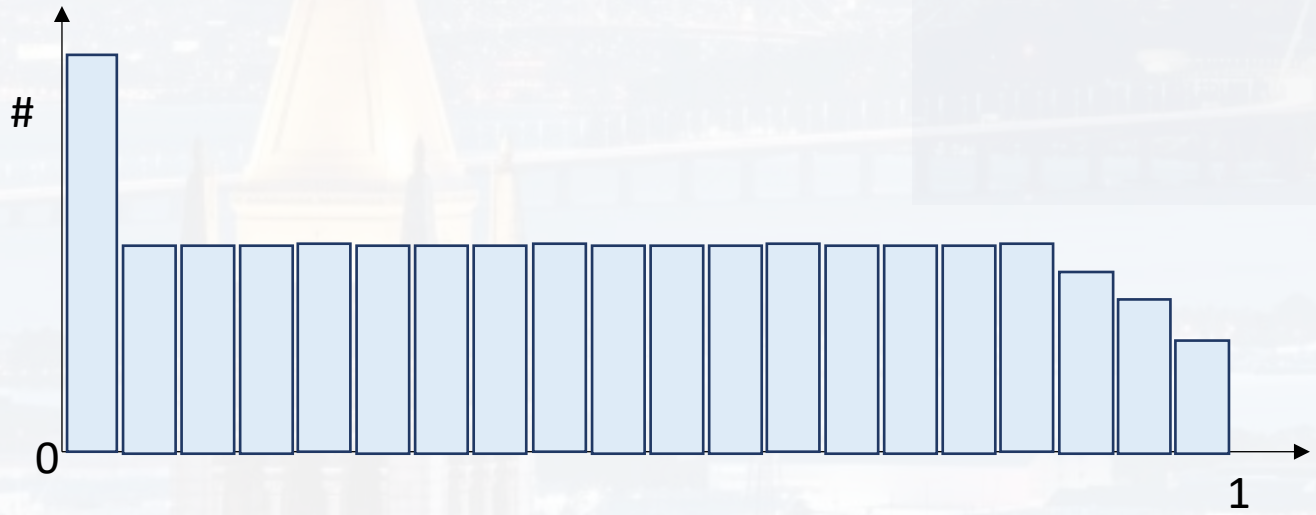
p-value $\boldsymbol{P(x \geq x_0 | H_0)}$

→ what does a **p-value histogram** look like if H_0 is **not always** true?

if H_0 is **not always** true:

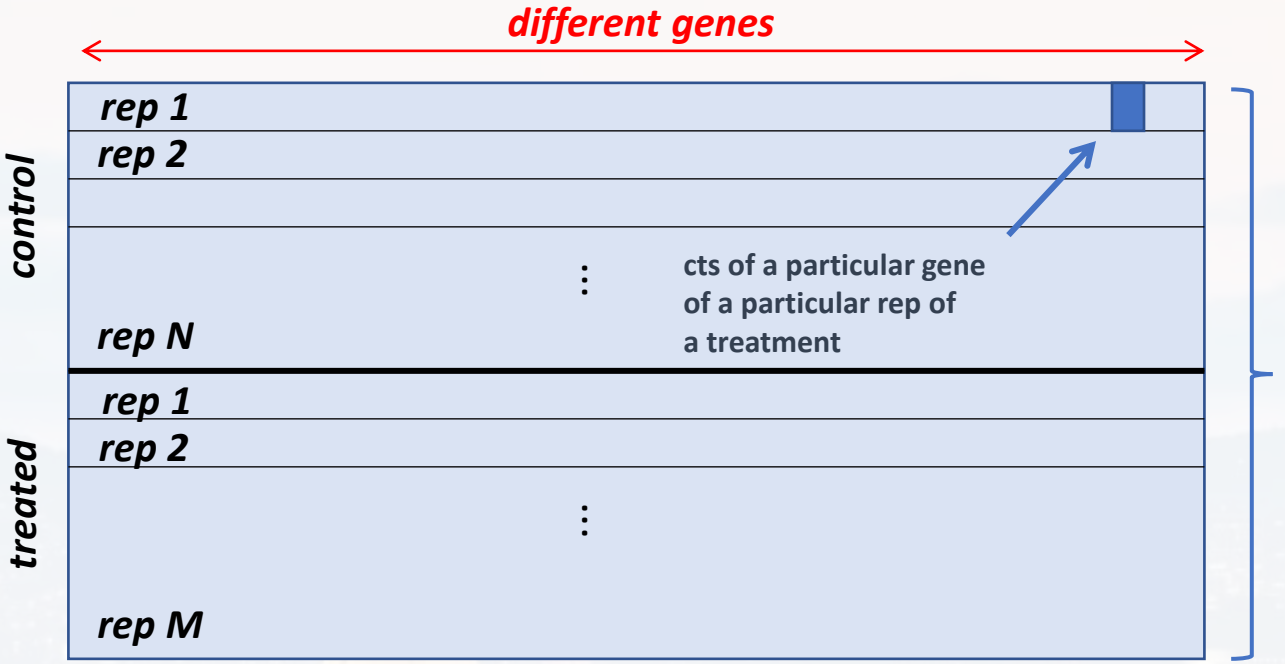
- we find more extreme values than expected
- smaller p-values are more frequent

→ **always check p-value distribution**



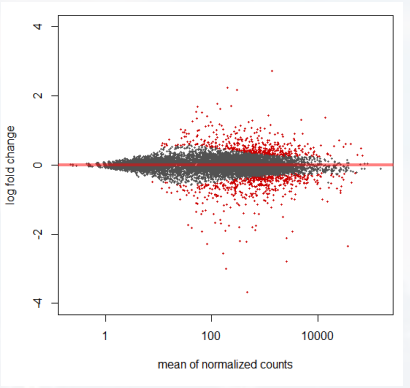
Feature Selection

More About p-values

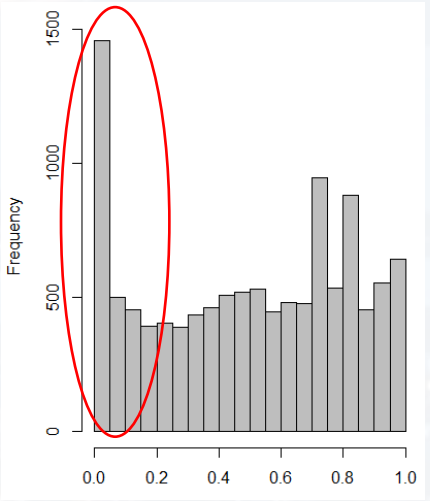


p-value $P(x \geq x_0 | H_0)$

log fold change for each gene
→ p-values for each gene



if treatment worked: some genes are highly over/under expressed
→ more smaller p-values than expected if H_0 is true

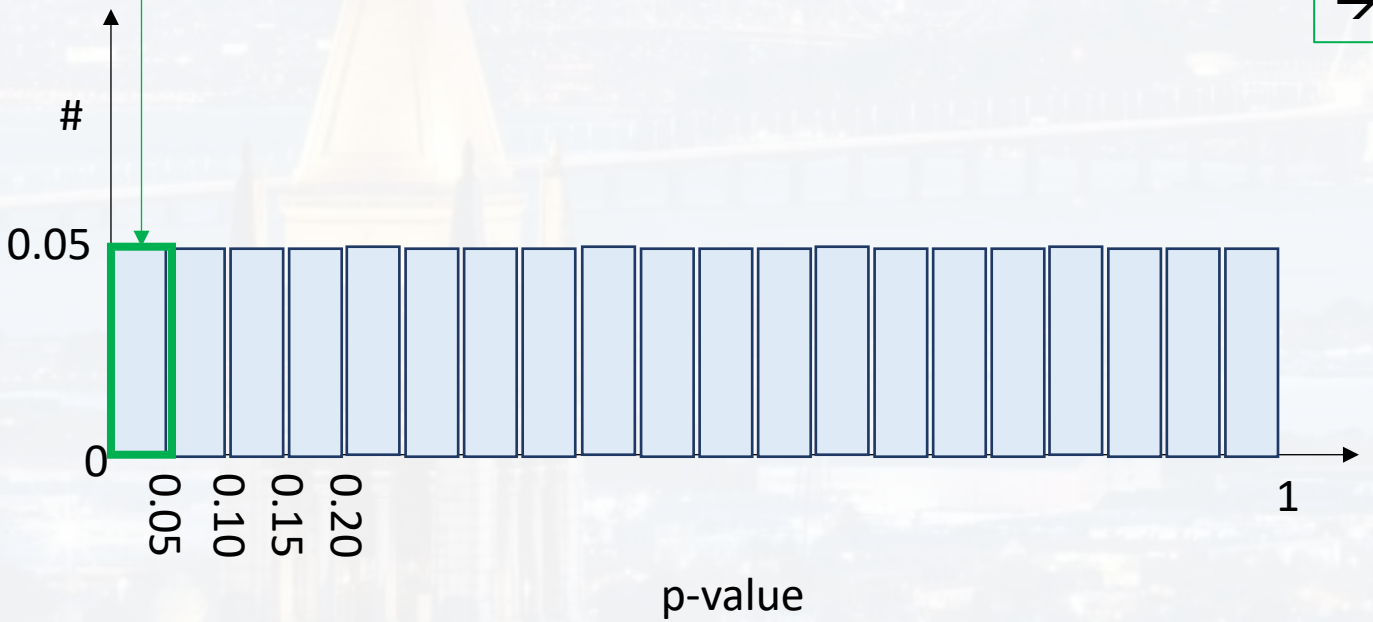


if H_0 is **always** true:

- the p-value distribution is flat!
- regardless of the test
- regardless of the data
- it is **the definition** of the p-value

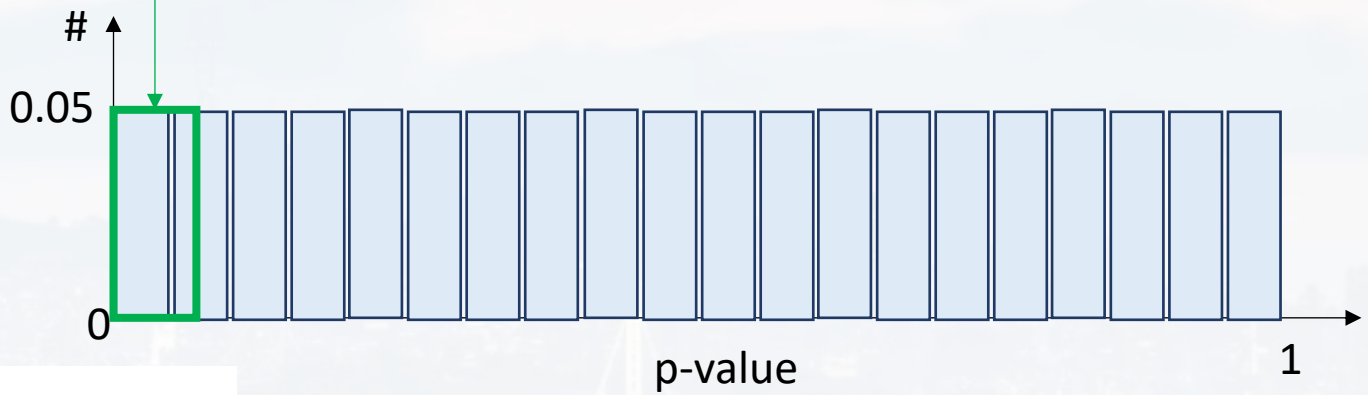
p-value $\boldsymbol{P(x \geq x_0 | H_0)}$

if $\alpha = 0.05$ then in 5% of the cases $\boldsymbol{H_0}$ gets rejected even though it is true
→ false positive rate!



if $\alpha = 0.XX$ then in **XX%** of the cases H_0 gets rejected even though it is true
→ false positive rate!

p-value $P(x \geq x_0 | H_0)$



note: we would reject H_0 **(= feature not relevant for the model)** for "Age" if $\alpha = 0.05$, but not for $\alpha = 0.01$!

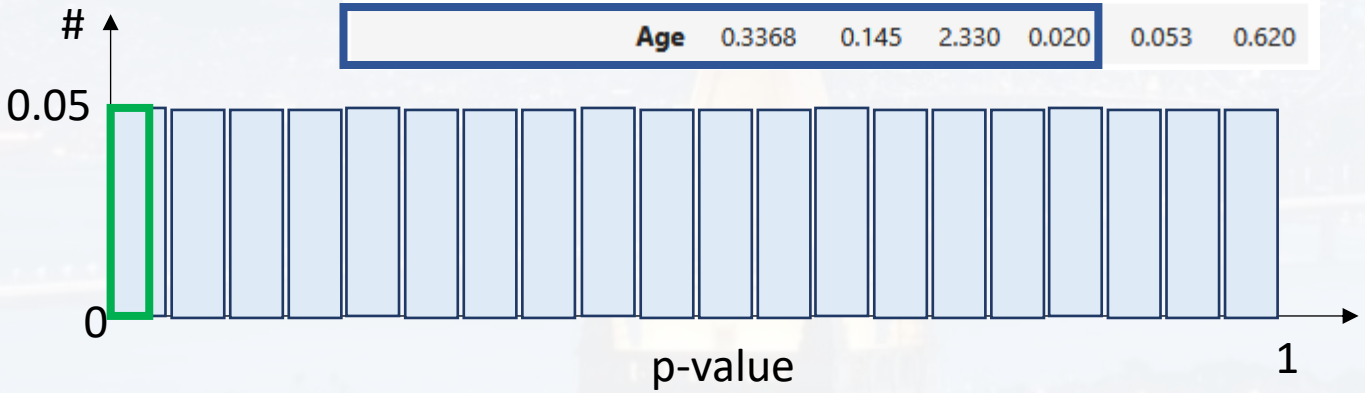
	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.9067	0.128	-7.087	0.000	-1.157	-0.656
Pregnancies	0.2391	0.134	1.786	0.074	-0.023	0.501
Glucose	1.0384	0.152	6.834	0.000	0.741	1.336
BloodPressure	-0.2400	0.139	-1.728	0.084	-0.512	0.032
SkinThickness	-0.0690	0.151	-0.457	0.648	-0.365	0.227
Insulin	0.0839	0.145	0.581	0.562	-0.199	0.367
BMI	0.8625	0.166	5.210	0.000	0.538	1.187
DiabetesPedigreeFunction	0.1161	0.126	0.923	0.356	-0.130	0.363
Age	0.3368	0.145	2.330	0.020	0.053	0.620

What is the probability to have p = 0.020, given H_0 is true if there are **nine p-values**?

What is the probability to have p = 0.074, given H_0 is true if there are nine p-values?

Feature Selection

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.9067	0.128	-7.087	0.000	-1.157	-0.656
Pregnancies	0.2391	0.134	1.786	0.074	-0.023	0.501
Glucose	1.0384	0.152	6.834	0.000	0.741	1.336
BloodPressure	-0.2400	0.139	-1.728	0.084	-0.512	0.032
SkinThickness	-0.0690	0.151	-0.457	0.648	-0.365	0.227
Insulin	0.0839	0.145	0.581	0.562	-0.199	0.367
BMI	0.8625	0.166	5.210	0.000	0.538	1.187
DiabetesPedigreeFunction	0.1161	0.126	0.923	0.356	-0.130	0.363
Age	0.3368	0.145	2.330	0.020	0.053	0.620



p-value $P(x \geq x_0 | H_0)$

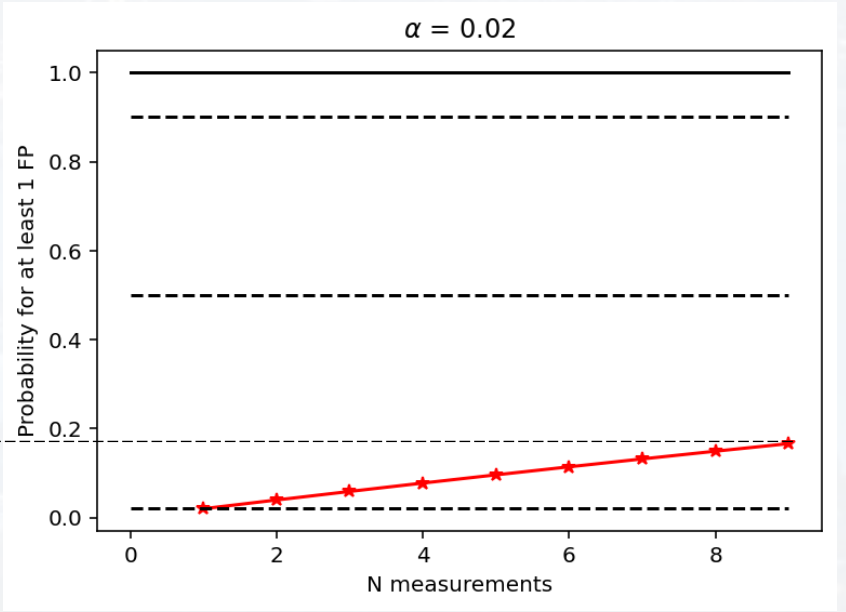
What is the probability to have at least one **p = 0.020**, given H_0 is true if there are nine p-values (= measurements)?

It is just a binomial problem! p = 0.02, 1-p = 0.98

`FalsePositive.py`

```
FalsePositive(N = 9, n = 1, alpha = 0.02)
```

about 20%!



Feature Selection

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.9067	0.128	-7.087	0.000	-1.157	-0.656
Pregnancies	0.2391	0.134	1.786	0.074	-0.023	0.501
Glucose	1.0384	0.152	6.834	0.000	0.741	1.336
BloodPressure	-0.2400	0.139	-1.728	0.084	-0.512	0.032
SkinThickness	-0.0690	0.151	-0.457	0.648	-0.365	0.227
Insulin	0.0839	0.145	0.581	0.562	-0.199	0.367
BMI	0.8625	0.166	5.210	0.000	0.538	1.187
DiabetesPedigreeFunction	0.1161	0.126	0.923	0.356	-0.130	0.363
Age	0.3368	0.145	2.330	0.020	0.053	0.620

p-value $P(x \geq x_0 | H_0)$

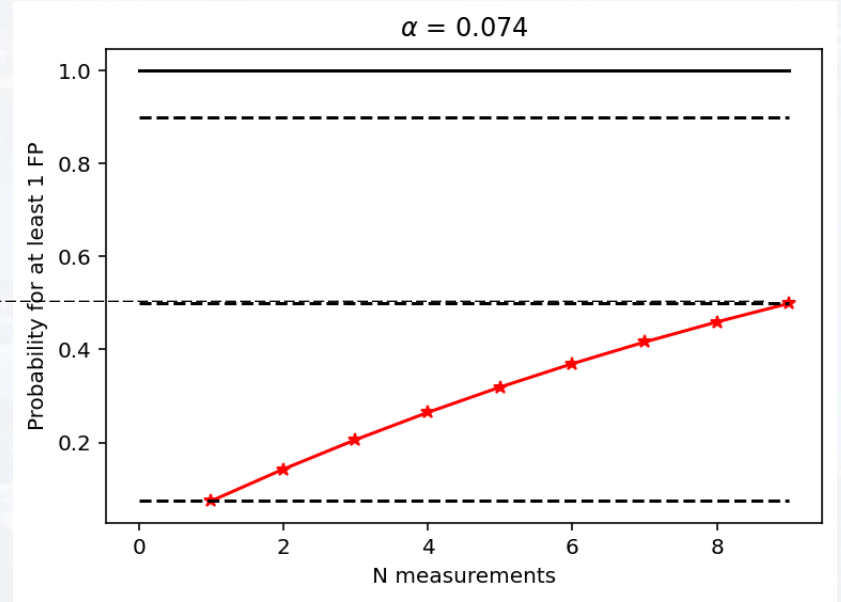
What is the probability to have at least one **p = 0.074**, given H_0 is true if there are nine p-values (= measurements)?

It is just a binomial problem!

`FalsePositive.py`

```
FalsePositive(N = 9, n = 1, alpha = 0.074)
```

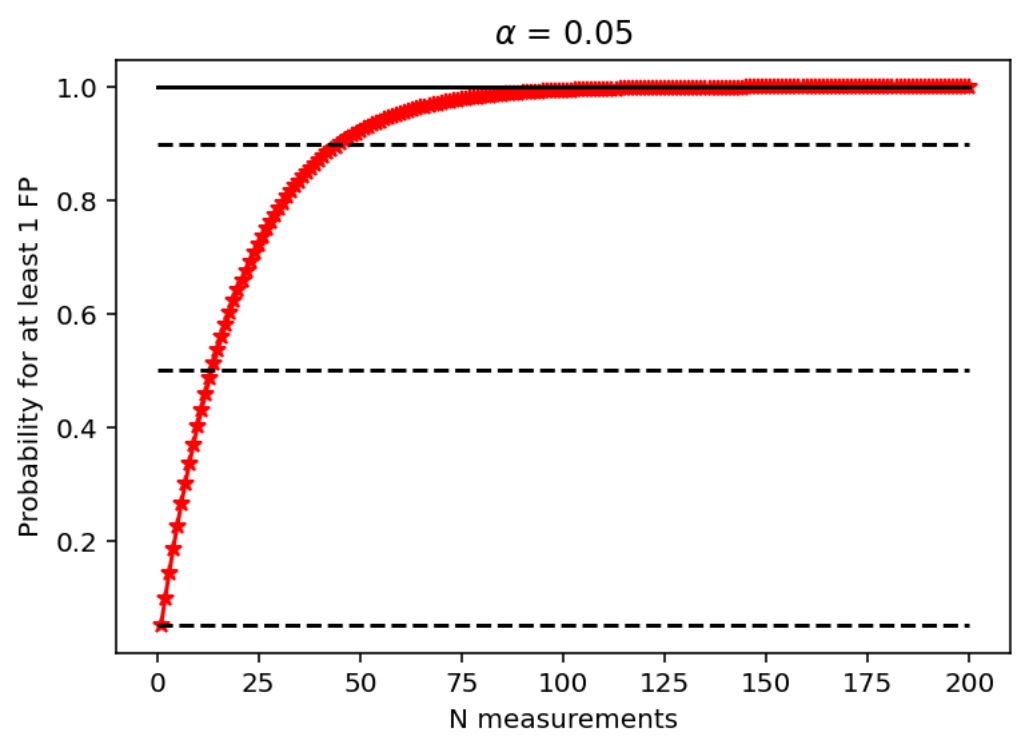
about 50%!



gene expression data: 200 different genes, p-values from log fold change

p-value $P(x \geq x_0 | H_0)$

```
FalsePositive(N = 200, n = 1, alpha = 0.05)
```

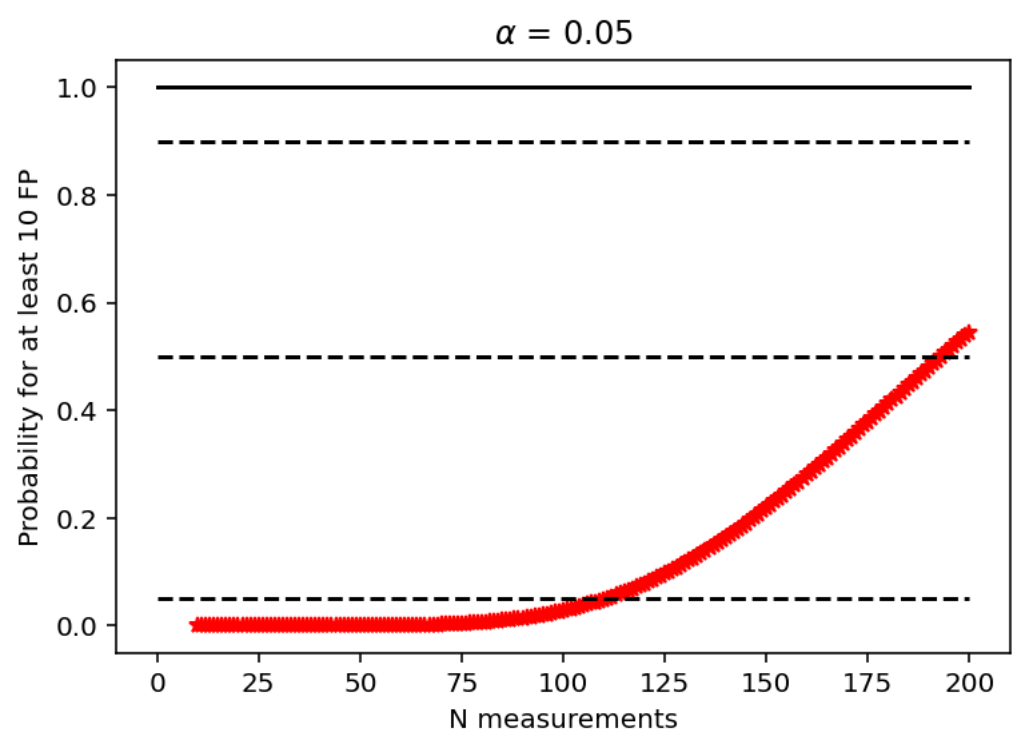


multiple testing problem!

gene expression data: 200 different genes, p-values from log fold change

p-value $P(x \geq x_0 | H_0)$

```
FalsePositive(N = 200, n = 10, alpha = 0.05)
```

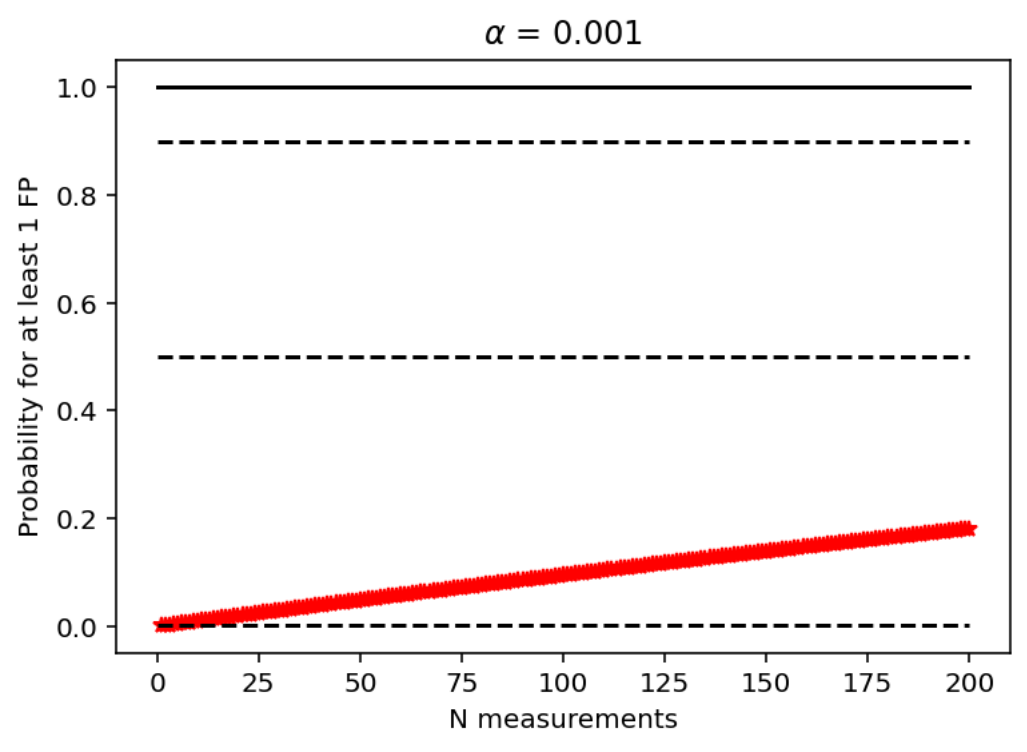


multiple testing problem!

gene expression data: 200 different genes, p-values from log fold change

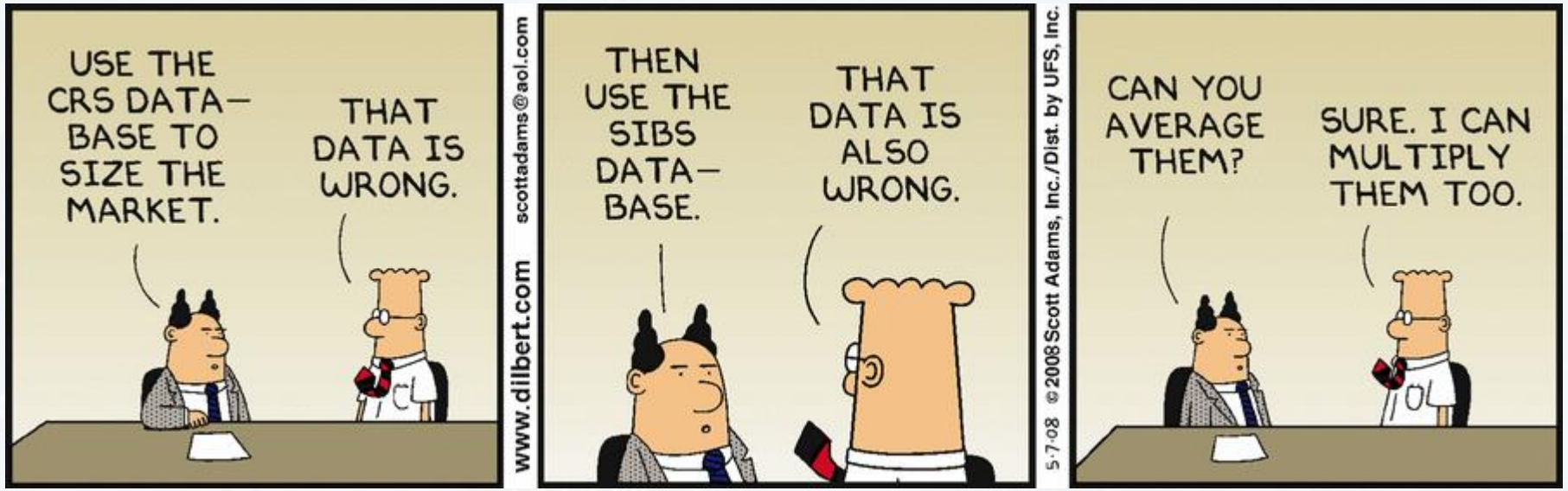
p-value $P(x \geq x_0 | H_0)$

```
FalsePositive(N = 200, n = 1, alpha = 0.001)
```



multiple testing problem!

Feature Selection



Outline

- More About p-values
- **Feature Selection**
  - via L1 and L2
  - via p-values
  - back to PCA

Feature Selection via L1 and L2

recall our best model for L1 and L2

'Diabetes2.csv'

X

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	label
0	6	148.0	72.0	35.00	184.42	33.6	0.63	50	diabetes
1	1	85.0	66.0	29.00	75.91	26.6	0.35	31	healthy
2	8	183.0	64.0	31.34	155.87	23.3	0.67	32	diabetes
3	1	89.0	66.0	23.00	94.00	28.1	0.17	21	healthy
4	0	137.0	40.0	35.00	168.00	43.1	2.29	33	diabetes

Y

```
alpha_1 = 0.085
alpha_2 = 0.85
```

conf 0.025	best	conf 0.975
-0.644043	-0.339928	-0.378195
0.000000	0.000000	0.276625
0.468804	0.598115	0.723600
-0.132069	0.000000	0.074831
0.000000	0.000000	0.207292
0.000000	0.000000	0.356646
0.287730	0.282465	0.525576
-0.034453	0.000000	0.235977
0.031497	0.000703	0.296114

"sparse" model

	Glucose	BloodPressure	BMI	DiabetesPedigreeFunction	Age
0	0.445311	0.122742	-0.877423	0.554441	1.481987
1	0.940272	-0.952811	-0.863201	-0.865148	-0.932781
2	-0.445618	-0.952811	-1.019650	-0.774536	-0.516442
3	-2.029492	0.737343	0.317280	-0.683924	1.065648
4	-0.214636	0.430042	0.559065	0.101380	-0.682978

```
best accuracy:  77.92%
```

How many (and which) features can we remove in order to maintain accuracy?

Feature Selection via L1 and L2

recall our best model for L1 and L2

'Diabetes2.csv'

X

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	label
0	6	148.0	72.0	35.00	184.42	33.6	0.63	50	diabetes
1	1	85.0	66.0	29.00	75.91	26.6	0.35	31	healthy
2	8	183.0	64.0	31.34	155.87	23.3	0.67	32	diabetes
3	1	89.0	66.0	23.00	94.00	28.1	0.17	21	healthy
4	0	137.0	40.0	35.00	168.00	43.1	2.29	33	diabetes

Y

```
alpha_1 = 0.085
alpha_2 = 0.85
```

conf 0.025	best	conf 0.975
-0.644043	-0.339928	-0.378195
0.000000	0.000000	0.276625
0.468804	0.598115	0.723600
-0.132069	0.000000	0.074831
0.000000	0.000000	0.207292
0.000000	0.000000	0.356646
0.287730	0.282465	0.525576
-0.034453	0.000000	0.235977
0.031497	0.000703	0.296114

removing all coefficients consistent with zero within their 2σ error bars

"sparse" model

	const	Glucose	BMI
0	1.0	0.445311	-0.877423
1	1.0	0.940272	-0.863201
2	1.0	-0.445618	-1.019650
3	1.0	-2.029492	0.317280
4	1.0	-0.214636	0.559065

Feature Selection via L1 and L2

no regularization

"sparse" model

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.8517	0.122	-6.966	0.000	-1.091	-0.612
Glucose	1.1150	0.133	8.398	0.000	0.855	1.375
BMI	0.7348	0.128	5.719	0.000	0.483	0.987

Accuracy is 75.97%

only those with small p-values "survived"

complete model

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.9067	0.128	-7.087	0.000	-1.157	-0.656
Pregnancies	0.2391	0.134	1.786	0.074	-0.023	0.501
Glucose	1.0384	0.152	6.834	0.000	0.741	1.336
BloodPressure	-0.2400	0.139	-1.728	0.084	-0.512	0.032
SkinThickness	-0.0690	0.151	-0.457	0.648	-0.365	0.227
Insulin	0.0839	0.145	0.581	0.562	-0.199	0.367
BMI	0.8625	0.166	5.210	0.000	0.538	1.187
DiabetesPedigreeFunction	0.1161	0.126	0.923	0.356	-0.130	0.363
Age	0.3368	0.145	2.330	0.020	0.053	0.620

Accuracy is 75.65%

L1 and L2 regularization

	conf 0.025	best	conf 0.975
const	-0.497288	-0.339920	-0.175653
Glucose	0.435954	0.598282	0.766013
BMI	0.117937	0.282437	0.431760

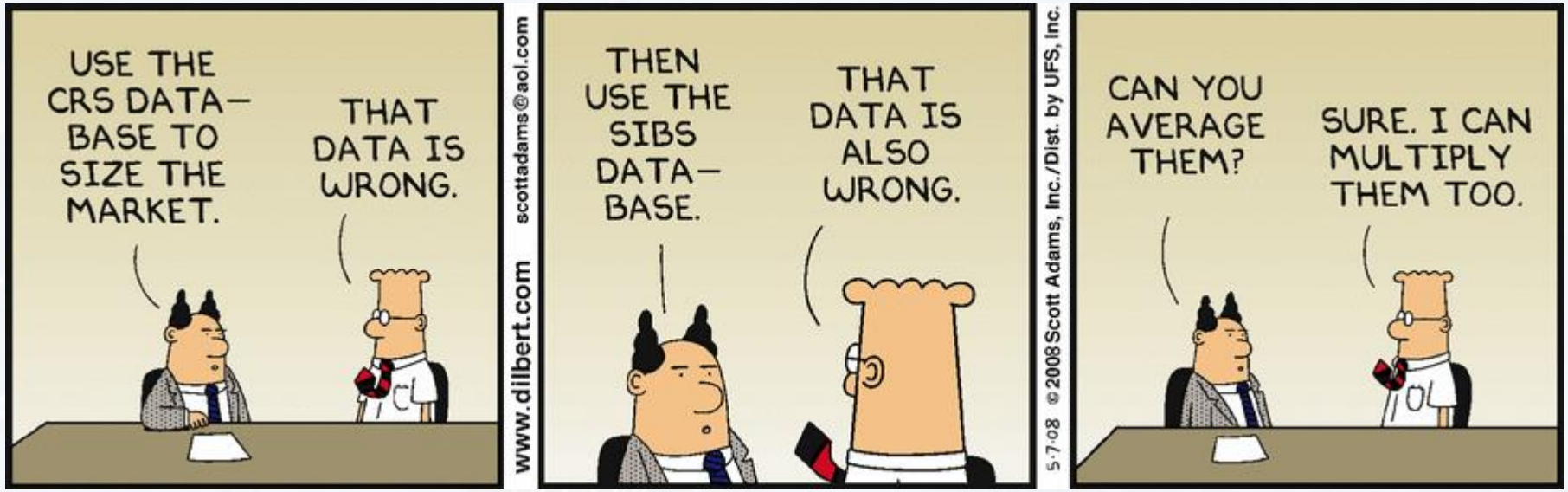
Accuracy is 77.92%

only those not consistent with zero "survived"

	conf 0.025	best	conf 0.975
const	-0.644043	-0.339928	-0.378195
Pregnancies	0.000000	0.000000	0.276625
Glucose	0.468804	0.598115	0.723600
BloodPressure	-0.132069	0.000000	0.074831
SkinThickness	0.000000	0.000000	0.207292
Insulin	0.000000	0.000000	0.356646
BMI	0.287730	0.282465	0.525576
DiabetesPedigreeFunction	-0.034453	0.000000	0.235977
Age	0.031497	0.000703	0.296114

Accuracy is 77.92%

Feature Selection



Outline

- More About p-values
- **Feature Selection**
  - via L1 and L2
  - via p-values
  - back to PCA

Feature Selection via p-values

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.9067	0.128	-7.087	0.000	-1.157	-0.656
Pregnancies	0.2391	0.134	1.786	0.074	-0.023	0.501
Glucose	1.0384	0.152	6.834	0.000	0.741	1.336
BloodPressure	-0.2400	0.139	-1.728	0.084	-0.512	0.032
SkinThickness	-0.0690	0.151	-0.457	0.648	-0.365	0.227
Insulin	0.0839	0.145	0.581	0.562	-0.199	0.367
BMI	0.8625	0.166	5.210	0.000	0.538	1.187
DiabetesPedigreeFunction	0.1161	0.126	0.923	0.356	-0.130	0.363
Age	0.3368	0.145	2.330	0.020	0.053	0.620

only those with small p-values "survived"

Apparently, keeping only those features with small p-values was a good strategy.

But as discussed, p-values can be tricky!

`for` loop

idea:

1) pick least significant feature **(according to p-value)**

2) run the model without that feature

3) evaluate the model, e.g. accuracy

4) go to 1)

finally, plot how accuracy changes when removing one feature after another

Feature Selection via p-values

no regularization

```
my_model = sm.GLM(Ydumm, XS_Train_add, family = sm.families.Binomial())
result   = my_model.fit()
```

running the model as before

```
result.pvalues
```

const	1.367712e-12
Pregnancies	7.415132e-02
Glucose	8.237812e-12
BloodPressure	8.401696e-02
SkinThickness	6.476258e-01
Insulin	5.615383e-01
BMI	1.887106e-07
DiabetesPedigreeFunction	3.558123e-01
Age	1.980355e-02

```
Nparam    = len(result.pvalues)
Accur     = np.zeros((Nparam + 1,))
Param     = [None]*Nparam
Param[0]  = 'none'
LenYTest  = len(Y_Test)
```

Keeping track of the features dropped and the change of accuracy

```
XTrain = XS_Train_add.copy()
XTest  = XS_Test_add.copy()
```

`lists` are mutable, therefore we use `copy()` in order to avoid changing the original data

Feature Selection via p-values

no regularization

```
for i in range(Nparam-1):

    my_model_i    = sm.GLM(Ydumm, XTrain, family = sm.families.Binomial())

    result_i      = my_model_i.fit()
```

running the model

evaluating the model

```
    Probs         = result_i.predict(XTest)
    ProbsR        = np.round(Probs)
    Ypred         = ['diabetes' if p==1 else 'healthy' for p in ProbsR]
```

storing accuracy

```
    Accur[i]      = 100*np.mean(Ypred == Y_Test)
```

finding and removing the least significant feature

```
    idx_drop      = np.argmax(result_i.pvalues)
    todrop        = XTrain.columns[idx_drop]

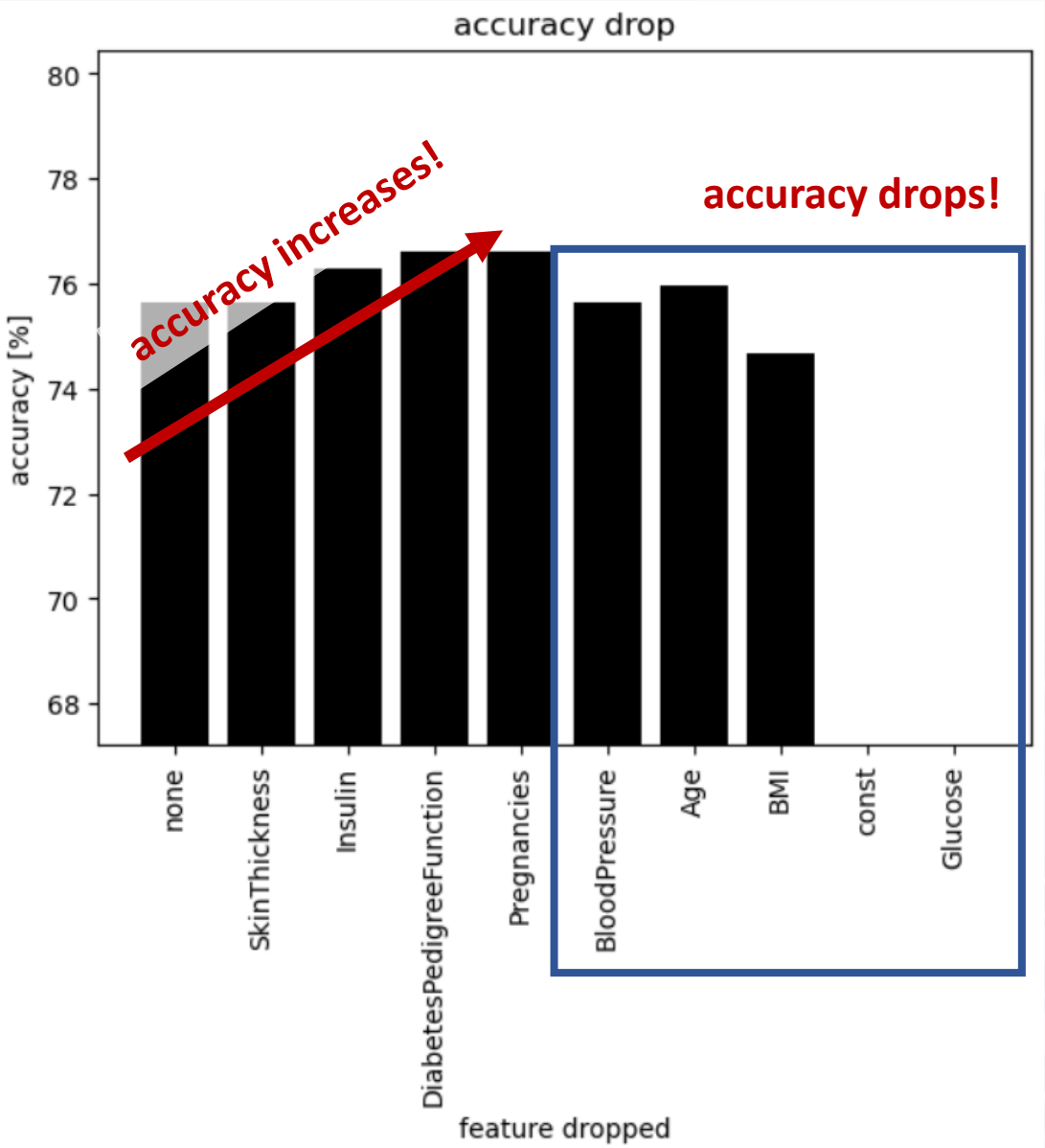
    Param[i+1]    = todrop
    XTrain        = XTrain.drop(todrop, axis = 1)
    XTest         = XTest.drop(todrop, axis = 1)
```

```
#check which parameter is left:
left = list(set(XS_Train_add.columns)-set(Param[1:]))
```

Feature Selection

via p-values

no regularization



	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.9067	0.128	-7.087	0.000	-1.157	-0.656
Pregnancies	0.2391	0.134	1.786	0.074	-0.023	0.501
Glucose	1.0384	0.152	6.834	0.000	0.741	1.336
BloodPressure	-0.2400	0.139	-1.728	0.084	-0.512	0.032
SkinThickness	-0.0690	0.151	-0.457	0.648	-0.365	0.227
Insulin	0.0839	0.145	0.581	0.562	-0.199	0.367
BMI	0.8625	0.166	5.210	0.000	0.538	1.187
DiabetesPedigreeFunction	0.1161	0.126	0.923	0.356	-0.130	0.363
Age	0.3368	0.145	2.330	0.020	0.053	0.620

"sparse" model

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.8517	0.122	-6.966	0.000	-1.091	-0.612
Glucose	1.1150	0.133	8.398	0.000	0.855	1.375
BMI	0.7348	0.128	5.719	0.000	0.483	0.987

Not entirely consistent with the sparse model!

Feature Selection

via p-values

L1 and L2 regularization

```
for i in range(Nparam-1):

        my_model_i    = sm.GLM(Ydumm, XTrain, family = sm.families.Binomial())

        result_i      = my_model_i.fit()


        Probs         = result_i.predict(XTest)
        ProbsR        = np.round(Probs)
        Ypred         = ['diabetes' if p==1 else 'healthy' for p in ProbsR]

        Accur[i]      = 100*np.mean(Ypred == Y_Test)

        idx_drop      = np.argmax(result_i.pvalues)
        todrop        = XTrain.columns[idx_drop]

        Param[i+1]    = todrop
        XTrain        = XTrain.drop(todrop, axis = 1)
        XTest         = XTest.drop(todrop, axis = 1)

#check which parameter is left:
left = list(set(XS_Train_add.columns)-set(Param[1:]))
```

Feature Selection

via p-values

L1 and L2 regularization

```
alpha_1 = 0.085
alpha_2 = 0.85
```

```
for i in range(Nparam-1):

        my_model_i    = sm.GLM(Ydumm, XTrain, family = sm.families.Binomial())

        result_i      = my_model_i.fit_regularized(method = 'elastic_net',
                                                   alpha = alpha1, L1_wt = alpha2)

        Probs         = result_i.predict(XTest)
        ProbsR        = np.round(Probs)
        Ypred         = ['diabetes' if p==1 else 'healthy' for p in ProbsR]

        Accur[i]      = 100*np.mean(Ypred == Y_Test)

        idx_drop      = np.argmax(result_i.pvalues)
        todrop        = XTrain.columns[idx_drop]

        Param[i+1]    = todrop
        XTrain        = XTrain.drop(todrop, axis = 1)
        XTest         = XTest.drop(todrop, axis = 1)

    #check which parameter is left:
    left = list(set(XS_Train_add.columns)-set(Param[1:]))
```

Feature Selection via p-values

L1 and L2 regularization

```
alpha_1 = 0.085
alpha_2 = 0.85
```

there are no p-values for L1 and L2

```
for i in range(Nparam-1):

        my_model_i    = sm.GLM(Ydumm, XTrain, family = sm.families.Binomial())

        result_i      = my_model_i.fit_regularized(method = 'elastic_net',
                                                   alpha = alpha1, L1_wt = alpha2)

        Probs         = result_i.predict(XTest)
        ProbsR        = np.round(Probs)
        Ypred         = ['diabetes' if p==1 else 'healthy' for p in ProbsR]

        Accur[i]      = 100*np.mean(Ypred == Y_Test)

        idx_drop      = np.argmin(abs(np.array(result_i.params)))
        todrop        = XTrain.columns[idx_drop]

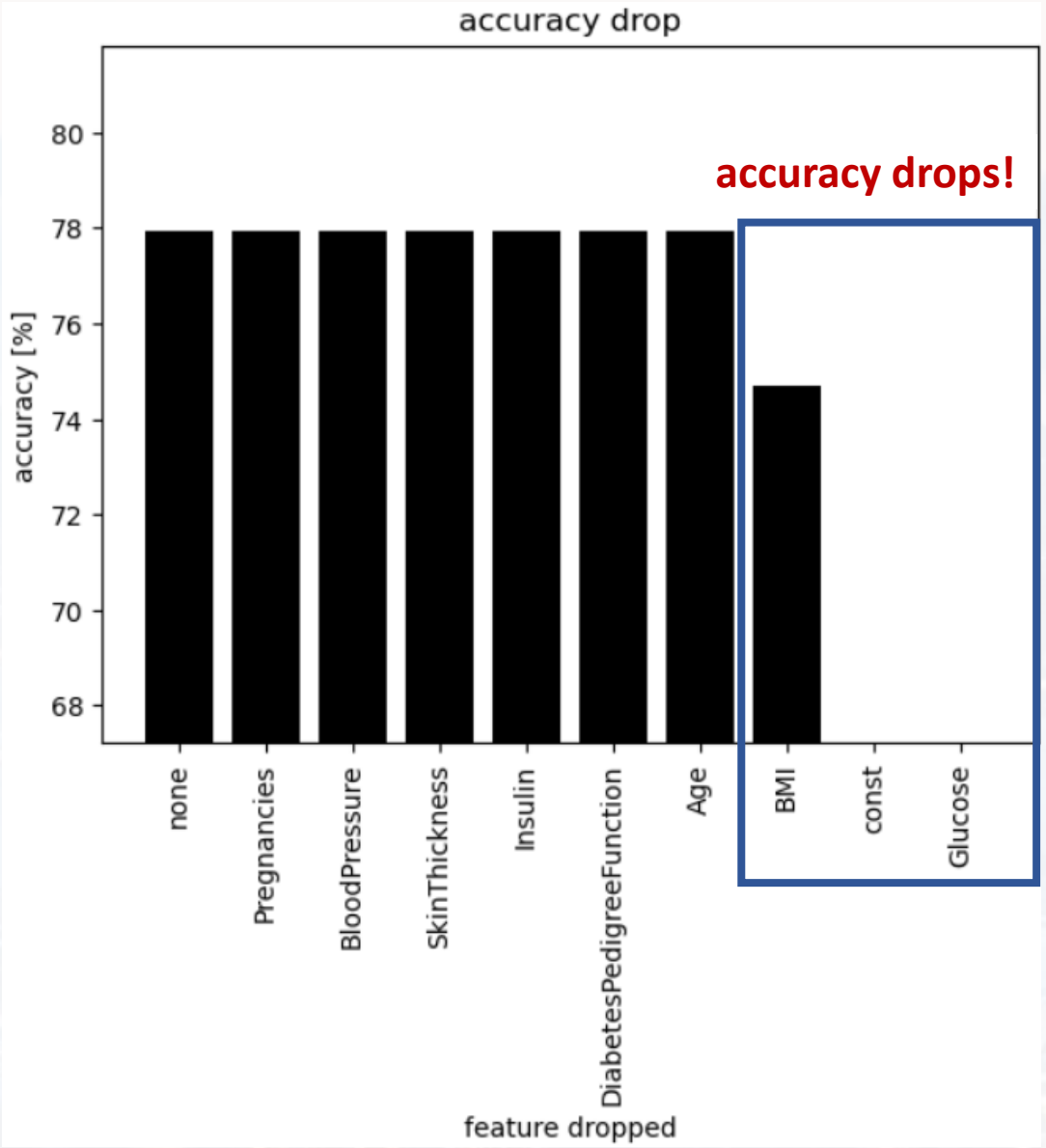
        Param[i+1]    = todrop
        XTrain        = XTrain.drop(todrop, axis = 1)
        XTest         = XTest.drop(todrop, axis = 1)

#check which parameter is left:
left = list(set(XS_Train_add.columns)-set(Param[1:]))
```

Feature Selection via p-values

L1 and L2 regularization

```
alpha_1 = 0.085
alpha_2 = 0.85
```



	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.9067	0.128	-7.087	0.000	-1.157	-0.656
Pregnancies	0.2391	0.134	1.786	0.074	-0.023	0.501
Glucose	1.0384	0.152	6.834	0.000	0.741	1.336
BloodPressure	-0.2400	0.139	-1.728	0.084	-0.512	0.032
SkinThickness	-0.0690	0.151	-0.457	0.648	-0.365	0.227
Insulin	0.0839	0.145	0.581	0.562	-0.199	0.367
BMI	0.8625	0.166	5.210	0.000	0.538	1.187
DiabetesPedigreeFunction	0.1161	0.126	0.923	0.356	-0.130	0.363
Age	0.3368	0.145	2.330	0.020	0.053	0.620

“sparse” model

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.8517	0.122	-6.966	0.000	-1.091	-0.612
Glucose	1.1150	0.133	8.398	0.000	0.855	1.375
BMI	0.7348	0.128	5.719	0.000	0.483	0.987

Entirely consistent with the sparse model!

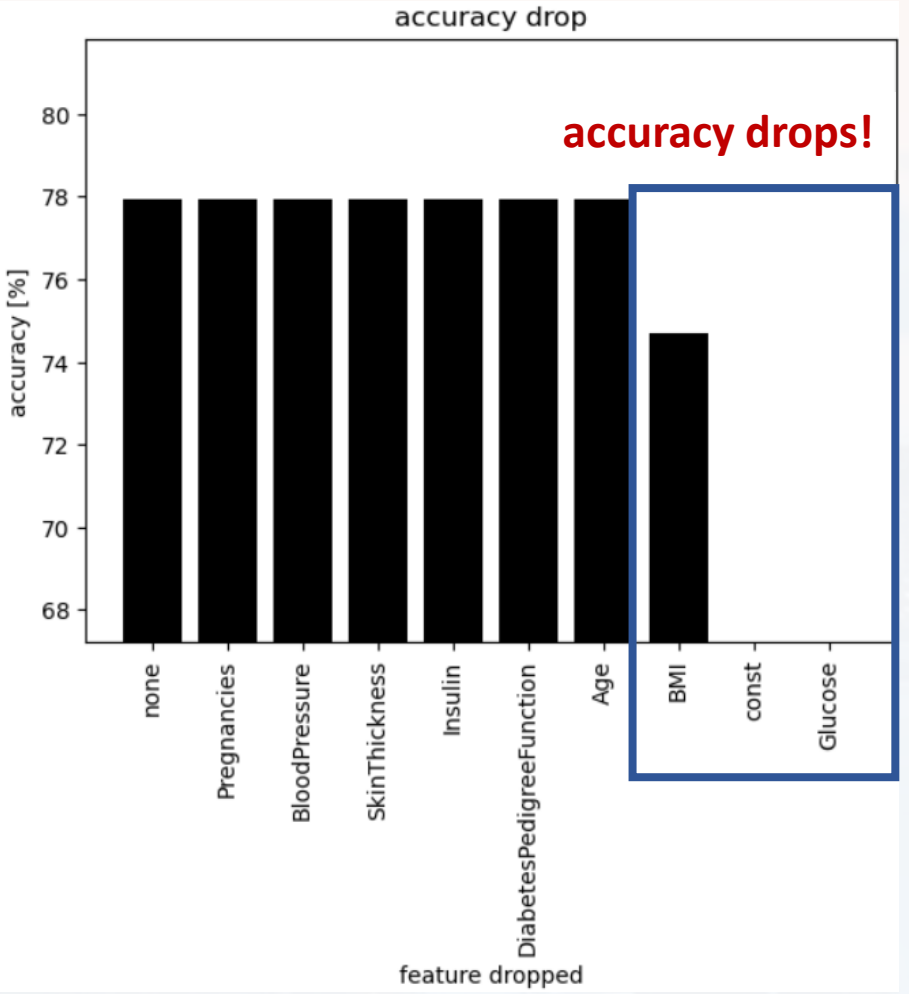
Feature Selection via p-values

L1 and L2 regularization

```
alpha_1 = 0.085
alpha_2 = 0.85
```

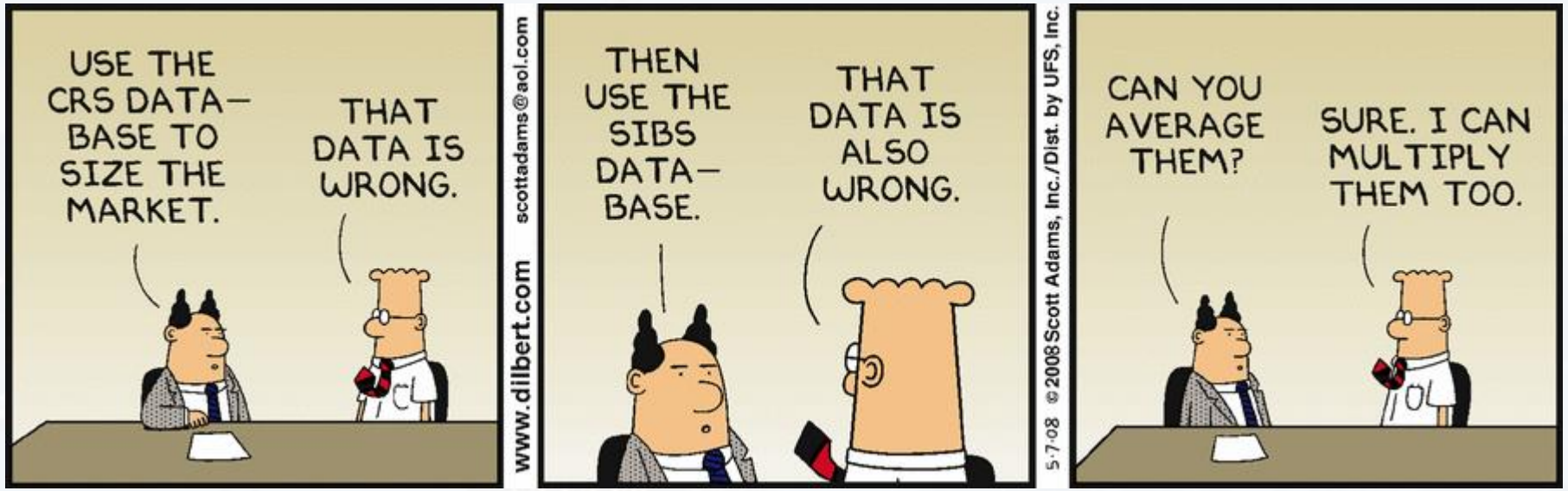
Summary:

- best model via L1 and L2
- supported by stepwise dropping features
- max accuracy 78%



	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.8517	0.122	-6.966	0.000	-1.091	-0.612
Glucose	1.1150	0.133	8.398	0.000	0.855	1.375
BMI	0.7348	0.128	5.719	0.000	0.483	0.987

Feature Selection



Outline

- More About p-values
- **Feature Selection**
  - via L1 and L2
  - via p-values
  - back to PCA

Feature Selection

back to PCA

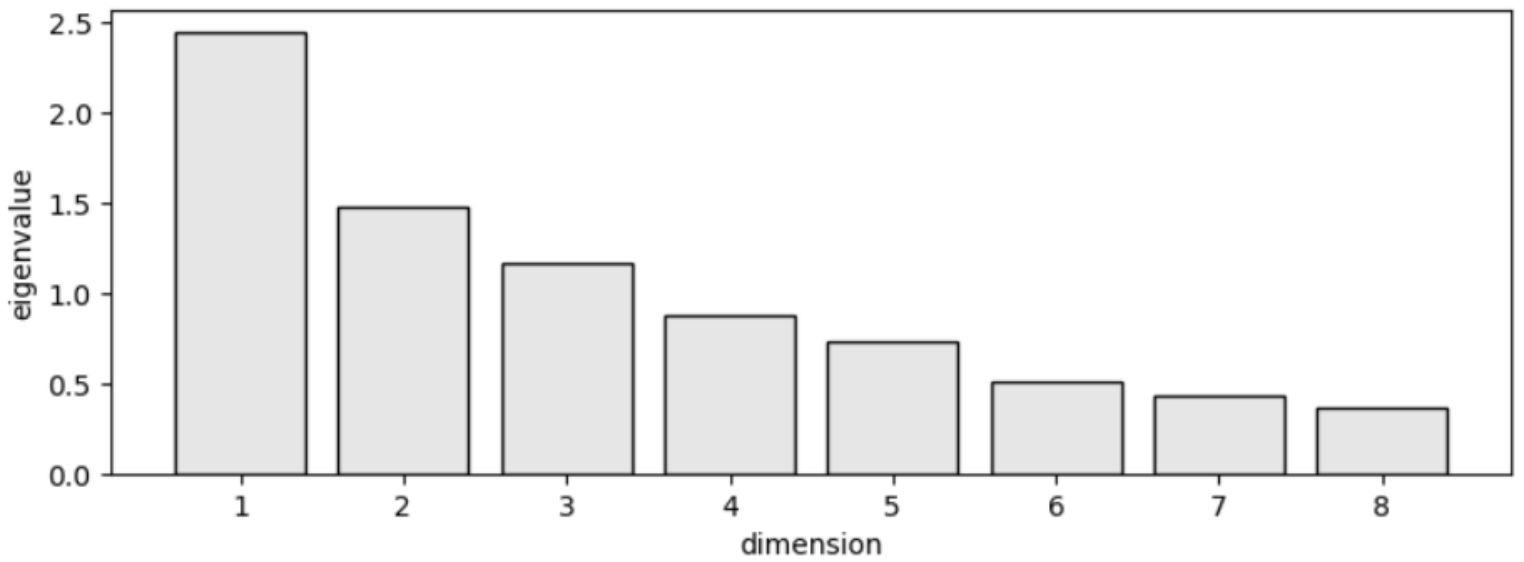
Didn't we also learn, that PCA (correlation analysis!) can be used for feature selection too?

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.8517	0.122	-6.966	0.000	-1.091	-0.612
Glucose	1.1150	0.133	8.398	0.000	0.855	1.375
BMI	0.7348	0.128	5.719	0.000	0.483	0.987

```
XS_Train = pd.DataFrame(XS_Train, columns = X.columns)
XS_Test  = pd.DataFrame(XS_Test, columns = X.columns)
N        = len(XS_Train.columns)

pca = PCA(n_components = N).fit(XS_Train)

eigenVal = pca.explained_variance_
```



Unfortunately, there is no sharp decrease in the eigenvalue spectrum!

Feature Selection

back to PCA

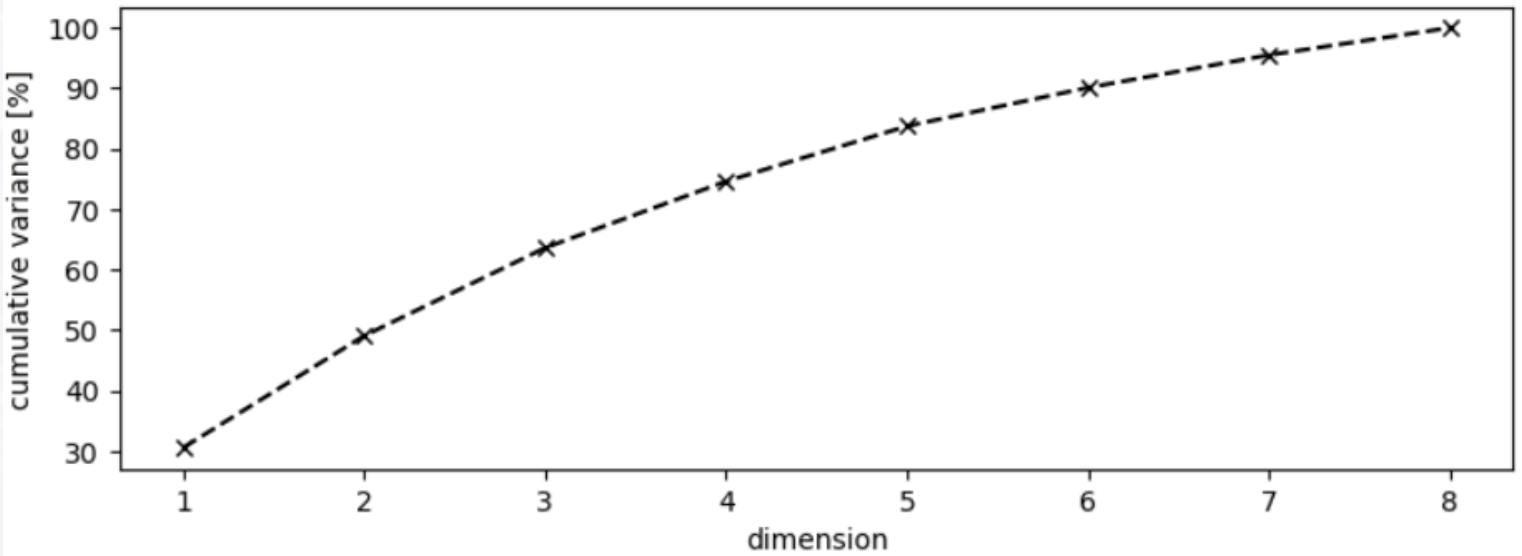
Didn't we also learn, that PCA (correlation analysis!) can be used for feature selection too?

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.8517	0.122	-6.966	0.000	-1.091	-0.612
Glucose	1.1150	0.133	8.398	0.000	0.855	1.375
BMI	0.7348	0.128	5.719	0.000	0.483	0.987

```
XS_Train = pd.DataFrame(XS_Train, columns = X.columns)
XS_Test  = pd.DataFrame(XS_Test, columns = X.columns)
N        = len(XS_Train.columns)

pca = PCA(n_components = N).fit(XS_Train)

eigenVal = pca.explained_variance_
```



No particular directions in the eigenspace stick out!

Feature Sele

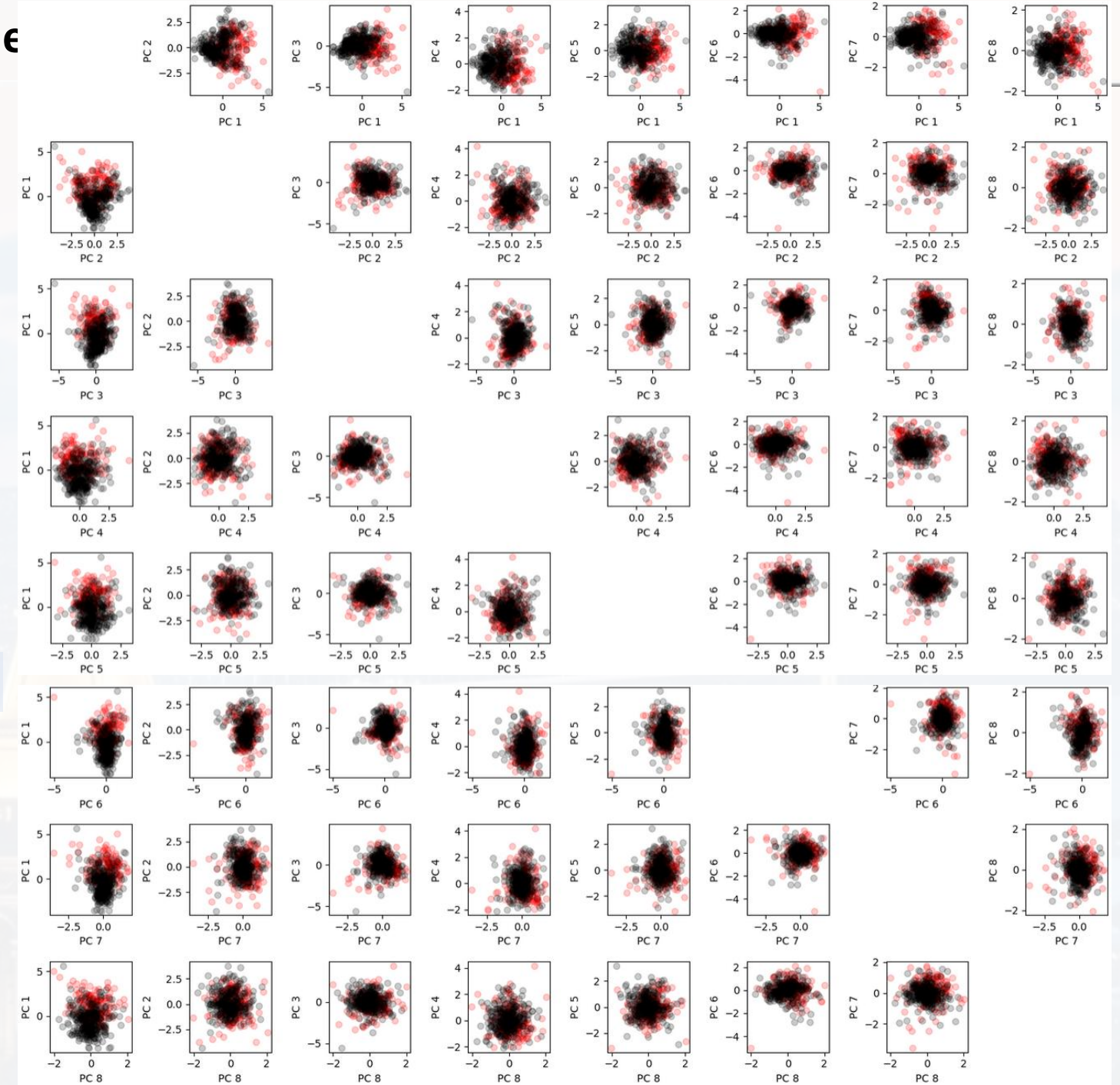
back to PCA

Clusters are not well separated in any of the directions!

PCA wouldn't have helped!

check out:

`FeatureSelection.ipynb`



Feature Selection

Thank you very much for your attention

